

# SalesDB – Transformations

```
USE SalesDB;
GO

-- Replace NULL customer last names
SELECT
    CustomerID,
    FirstName,
    COALESCE(LastName, 'Unknown') AS LastName,
    Country,
    Score
FROM Sales.Customers;
GO

-- Replace NULL scores with 0
SELECT
    CustomerID,
    FirstName,
    LastName,
    Country,
    COALESCE(Score, 0) AS Score
FROM Sales.Customers;
GO

-- Handle missing last names
SELECT
    CustomerID,
    CONCAT(
        FirstName,
        ' ',
        COALESCE(LastName, '')
    ) AS FullName
FROM Sales.Customers;
GO

-- Standardize country names
SELECT
    CustomerID,
    Country,
    CASE
        WHEN Country IN ('USA', 'US', 'United States')
            THEN 'United States'
        WHEN Country IN ('Germany', 'DE')
            THEN 'Germany'
        ELSE Country
    END AS Country
FROM Sales.Customers;
GO

-- Create customer full name with COALESCE
SELECT
    CustomerID,
    CONCAT(
        FirstName,
        ' ',
        COALESCE(LastName, 'Unknown')
    ) AS FullName
FROM Sales.Customers;
GO

-- Generate customer key
```

```

SELECT
    CustomerID,
    CONCAT('CUST-', RIGHT('00000' + CAST(CustomerID AS VARCHAR(5)), 5))
        AS CustomerKey
FROM Sales.Customers;
GO

```

-- Flag missing information

```

SELECT
    CustomerID,
    CASE
        WHEN FirstName IS NULL THEN 1
        WHEN LastName IS NULL THEN 1
        WHEN Country IS NULL THEN 1
        WHEN Score IS NULL THEN 1
        ELSE 0
    END AS HasMissingData
FROM Sales.Customers;
GO

```

-- Standardize gender

```

SELECT
    EmployeeID,
    CASE
        WHEN Gender IN ('M', 'm') THEN 'Male'
        WHEN Gender IN ('F', 'f') THEN 'Female'
        ELSE 'Unknown'
    END AS Gender
FROM Sales.Employees;
GO

```

-- Create salary category

```

SELECT
    EmployeeID,
    Salary,
    CASE
        WHEN Salary >= 100000 THEN 'High'
        WHEN Salary >= 50000 THEN 'Medium'
        ELSE 'Low'
    END AS SalaryCategory
FROM Sales.Employees;
GO

```

-- Calculate employee age

```

SELECT
    EmployeeID,
    BirthDate,
    DATEDIFF(YEAR, BirthDate, GETDATE()) AS Age
FROM Sales.Employees;
GO

```

-- Create age group

```

SELECT
    EmployeeID,
    BirthDate,
    CASE
        WHEN DATEDIFF(YEAR, BirthDate, GETDATE()) < 30
            THEN 'Under 30'
        WHEN DATEDIFF(YEAR, BirthDate, GETDATE()) < 40
            THEN '30-39'
        WHEN DATEDIFF(YEAR, BirthDate, GETDATE()) < 50
            THEN '40-49'
        ELSE '50+'
    END AS AgeGroup

```

```
FROM Sales.Employees;  
GO
```

```
-- Calculate annual salary  
SELECT  
    EmployeeID,  
    Salary,  
    Salary * 12 AS AnnualSalary  
FROM Sales.Employees;  
GO
```

```
-- Identify managers  
SELECT  
    EmployeeID,  
    FirstName,  
    LastName,  
    CASE  
        WHEN ManagerID IS NULL THEN 'Manager'  
        ELSE 'Employee'  
    END AS EmployeeType  
FROM Sales.Employees;  
GO
```

```
-- Enrich orders with customer name  
SELECT  
    o.OrderID,  
    o.CustomerID,  
    CONCAT(c.FirstName, ' ', COALESCE(c.LastName, '')) AS CustomerName,  
    o.OrderDate,  
    o.Sales  
FROM Sales.Orders o  
LEFT JOIN Sales.Customers c  
    ON o.CustomerID = c.CustomerID;  
GO
```

```
-- Add customer country to orders  
SELECT  
    o.OrderID,  
    o.OrderDate,  
    o.Sales,  
    c.Country  
FROM Sales.Orders o  
LEFT JOIN Sales.Customers c  
    ON o.CustomerID = c.CustomerID;  
GO
```

```
-- Add employee information  
SELECT  
    o.OrderID,  
    o.EmployeeID,  
    CONCAT(e.FirstName, ' ', e.LastName) AS EmployeeName  
FROM Sales.Orders o  
LEFT JOIN Sales.Employees e  
    ON o.EmployeeID = e.EmployeeID;  
GO
```

```
-- Create YYYY-MM  
SELECT  
    OrderID,  
    OrderDate,  
    FORMAT(OrderDate, 'yyyy-MM') AS YearMonth  
FROM Sales.Orders;  
GO
```

```

-- Create month name
SELECT
    OrderID,
    DATENAME(MONTH, OrderDate) AS MonthName
FROM Sales.Orders;
GO

```

```

-- Monthly sales
SELECT
    YEAR(OrderDate) AS OrderYear,
    MONTH(OrderDate) AS OrderMonth,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY
    YEAR(OrderDate),
    MONTH(OrderDate);
GO

```

```

-- Daily sales
SELECT
    CAST(OrderDate AS DATE) AS OrderDay,
    SUM(Sales) AS TotalSales
FROM Sales.Orders
GROUP BY CAST(OrderDate AS DATE);
GO

```

```

-- Sales by country
SELECT
    c.Country,
    SUM(o.Sales) AS TotalSales
FROM Sales.Orders o
JOIN Sales.Customers c
    ON o.CustomerID = c.CustomerID
GROUP BY c.Country;
GO

```

```

-- Rank customers by revenue
WITH CustomerSales AS
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalRevenue
    FROM Sales.Orders
    GROUP BY CustomerID
)
SELECT
    *,
    RANK() OVER (
        ORDER BY TotalRevenue DESC
    ) AS RevenueRank
FROM CustomerSales;
GO

```

```

-- Dense rank customers
WITH CustomerSales AS
(
    SELECT
        CustomerID,
        SUM(Sales) AS TotalRevenue
    FROM Sales.Orders
    GROUP BY CustomerID
)

```

```

SELECT
    *,
    DENSE_RANK() OVER (
        ORDER BY TotalRevenue DESC
    ) AS RevenueRank
FROM CustomerSales;
GO

```

-- Previous order date

```

SELECT
    OrderID,
    CustomerID,
    OrderDate,

    LAG(OrderDate) OVER (
        PARTITION BY CustomerID
        ORDER BY OrderDate
    ) AS PreviousOrderDate

FROM Sales.Orders;
GO

```

-- Next order date

```

SELECT
    OrderID,
    CustomerID,
    OrderDate,

    LEAD(OrderDate) OVER (
        PARTITION BY CustomerID
        ORDER BY OrderDate
    ) AS NextOrderDate

FROM Sales.Orders;
GO

```

-- Days since previous order

```

WITH x AS
(
    SELECT
        OrderID,
        CustomerID,
        OrderDate,

        LAG(OrderDate) OVER (
            PARTITION BY CustomerID
            ORDER BY OrderDate
        ) AS PreviousOrderDate

    FROM Sales.Orders
)
SELECT
    *,
    DATEDIFF(
        DAY,
        PreviousOrderDate,
        OrderDate
    ) AS DaysSincePreviousOrder
FROM x;
GO

```

-- Running customer revenue

```

SELECT
    OrderID,

```

```

CustomerID,
OrderDate,
Sales,

SUM(Sales) OVER (
    PARTITION BY CustomerID
    ORDER BY OrderDate
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND CURRENT ROW
) AS RunningRevenue

FROM Sales.Orders;
Go

```

```

-- Running total company revenue
SELECT
    OrderID,
    OrderDate,
    Sales,

    SUM(Sales) OVER (
        ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING
        AND CURRENT ROW
    ) AS RunningRevenue

FROM Sales.Orders;
GO

```

```

-- First order per customer
WITH x AS
(
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY CustomerID
            ORDER BY OrderDate
        ) AS rn
    FROM Sales.Orders
)
SELECT *
FROM x
WHERE rn = 1;
GO

```

```

-- Latest order per customer
WITH x AS
(
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY CustomerID
            ORDER BY OrderDate DESC
        ) AS rn
    FROM Sales.Orders
)
SELECT *
FROM x
WHERE rn = 1;
GO

```

```

-- Customer lifetime
SELECT
    CustomerID,
    DATEDIFF(

```

```

        DAY,
        MIN(OrderDate),
        MAX(OrderDate)
    ) AS CustomerLifetimeDays
FROM Sales.Orders
GROUP BY CustomerID;
Go

```

```

-- Customer recency
SELECT
    CustomerID,
    DATEDIFF(
        DAY,
        MAX(OrderDate),
        GETDATE()
    ) AS DaysSinceLastOrder
FROM Sales.Orders
GROUP BY CustomerID;
GO

```

```

-- Detect duplicate customers
SELECT
    FirstName,
    LastName,
    Country,
    COUNT(*) AS DuplicateCount
FROM Sales.Customers
GROUP BY
    FirstName,
    LastName,
    Country
HAVING COUNT(*) > 1;
Go

```

```

-- Detect duplicate orders
SELECT
    CustomerID,
    OrderDate,
    Sales,
    COUNT(*) AS DuplicateCount
FROM Sales.Orders
GROUP BY
    CustomerID,
    OrderDate,
    Sales
HAVING COUNT(*) > 1;
GO

```

```

-- Find negative sales
SELECT *
FROM Sales.Orders
WHERE Sales < 0;
GO

```

```

-- Create sales validity flag
SELECT
    *,
    CASE
        WHEN Sales IS NULL THEN 'Missing'
        WHEN Sales < 0 THEN 'Invalid'
        WHEN Sales = 0 THEN 'Zero'
        ELSE 'Valid'
    END AS SalesStatus

```

```
FROM Sales.Orders;
GO
```

```
-- Remove duplicate customers using ROW_NUMBER
```

```
WITH x AS
(
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY
                FirstName,
                LastName,
                Country
            ORDER BY CustomerID
        ) AS rn
    FROM Sales.Customers
)
SELECT *
FROM x
WHERE rn = 1;
GO
```

```
-- Monthly customer revenue
```

```
SELECT
    CustomerID,

    DATEFROMPARTS(
        YEAR(OrderDate),
        MONTH(OrderDate),
        1
    ) AS MonthStart,

    SUM(Sales) AS MonthlyRevenue

FROM Sales.Orders

GROUP BY
    CustomerID,
    DATEFROMPARTS(
        YEAR(OrderDate),
        MONTH(OrderDate),
        1
    );
GO
```

```
-- Month-over-month revenue
```

```
WITH MonthlySales AS
(
    SELECT
        DATEFROMPARTS(
            YEAR(OrderDate),
            MONTH(OrderDate),
            1
        ) AS MonthStart,

        SUM(Sales) AS Revenue

    FROM Sales.Orders

    GROUP BY
        DATEFROMPARTS(
            YEAR(OrderDate),
            MONTH(OrderDate),
            1
        )
)
)
```

```

SELECT
    MonthStart,
    Revenue,

    LAG(Revenue) OVER (
        ORDER BY MonthStart
    ) AS PreviousMonthRevenue,

    Revenue -
    LAG(Revenue) OVER (
        ORDER BY MonthStart
    ) AS RevenueChange

FROM MonthlySales;
GO

-- Create customer analytical table
WITH CustomerSales AS
(
    SELECT
        CustomerID,

        MIN(OrderDate) AS FirstOrderDate,
        MAX(OrderDate) AS LastOrderDate,

        COUNT(OrderID) AS TotalOrders,

        SUM(Sales) AS TotalRevenue,

        AVG(Sales) AS AverageOrderValue,

        MIN(Sales) AS MinimumOrderValue,

        MAX(Sales) AS MaximumOrderValue

    FROM Sales.Orders

    GROUP BY CustomerID
)
SELECT
    c.CustomerID,

    CONCAT(
        c.FirstName,
        ' ',
        COALESCE(c.LastName, 'Unknown')
    ) AS CustomerName,

    c.Country,

    COALESCE(c.Score, 0) AS CustomerScore,

    cs.FirstOrderDate,
    cs.LastOrderDate,

    cs.TotalOrders,
    cs.TotalRevenue,
    cs.AverageOrderValue,

    DATEDIFF(
        DAY,
        cs.FirstOrderDate,
        cs.LastOrderDate
    ) AS CustomerLifetimeDays,

    DATEDIFF(
        DAY,
        cs.LastOrderDate,

```

```

        GETDATE()
    ) AS DaysSinceLastOrder,

    CASE
        WHEN cs.TotalRevenue >= 10000
            THEN 'VIP'

        WHEN cs.TotalRevenue >= 5000
            THEN 'High Value'

        WHEN cs.TotalRevenue >= 1000
            THEN 'Medium Value'

        ELSE 'Low Value'
    END AS CustomerValueSegment

FROM Sales.Customers c

LEFT JOIN CustomerSales cs
    ON c.CustomerID = cs.CustomerID;
GO

-- Build a complete analytical customer dimension
WITH CustomerSales AS
(
    SELECT
        CustomerID,

        MIN(OrderDate) AS FirstOrderDate,
        MAX(OrderDate) AS LastOrderDate,

        COUNT(OrderID) AS TotalOrders,

        SUM(Sales) AS TotalRevenue,

        AVG(Sales) AS AverageOrderValue

    FROM Sales.Orders

    GROUP BY CustomerID
),

CustomerAnalytics AS
(
    SELECT
        c.CustomerID,

        CONCAT(
            TRIM(c.FirstName),
            ', ',
            COALESCE(TRIM(c.LastName), 'Unknown')
        ) AS CustomerName,

        c.Country,

        CASE
            WHEN c.Country = 'USA'
                THEN 'North America'

            WHEN c.Country = 'Germany'
                THEN 'Europe'

            ELSE 'Other'
        END AS Region,

        COALESCE(c.Score, 0) AS CustomerScore,

        cs.FirstOrderDate,

```

```

        cs.LastOrderDate,

        COALESCE(cs.TotalOrders, 0) AS TotalOrders,

        COALESCE(cs.TotalRevenue, 0) AS TotalRevenue,

        COALESCE(cs.AverageOrderValue, 0)
        AS AverageOrderValue

FROM Sales.Customers c

LEFT JOIN CustomerSales cs
ON c.CustomerID = cs.CustomerID
)

SELECT
    *,

    CASE
        WHEN TotalRevenue >= 10000
            THEN 'VIP'

        WHEN TotalRevenue >= 5000
            THEN 'High Value'

        WHEN TotalRevenue >= 1000
            THEN 'Medium Value'

        WHEN TotalRevenue > 0
            THEN 'Low Value'

        ELSE 'No Purchase'
    END AS CustomerValueSegment,

    CASE
        WHEN LastOrderDate IS NULL
            THEN 'Never Purchased'

        WHEN DATEDIFF(
            DAY,
            LastOrderDate,
            GETDATE()
        ) <= 30
            THEN 'Active'

        WHEN DATEDIFF(
            DAY,
            LastOrderDate,
            GETDATE()
        ) <= 90
            THEN 'At Risk'

        ELSE 'Churned'
    END AS CustomerStatus,

    CASE
        WHEN TotalOrders = 0
            THEN NULL

        ELSE
            TotalRevenue / TotalOrders
    END AS CalculatedAverageOrderValue

FROM CustomerAnalytics;
Go

```